

VEHICLE RENTAL MANAGEMENT SYSTEM

PROJECT REPORT
DBMS

SECTION: 2

FACULTY : PROF. PAVAN KUMAR SIR

Team Details

Name of the member	Roll
Thota Prem Kumar	S20240010243
Tellakula Chaitanya Naga Sai Pavan	S20240010237
Shameer Basha Shaik	S20240010222
Hemanth Kumar Avunuri	S20240010018
Gunda Naga Venkata Ghana Sathwik	S20240030370

Intro

Our DBMS Project, the Vehicle Rental Management System is designed to automate and streamline the process of renting vehicles to customers efficiently.

It maintains detailed records of customers, vehicles, bookings, payments, and feedback in a centralized database.

This system ensures accurate data handling, quick retrieval of information, and improved customer service through effective database management.

Problem Statement

Managing vehicle rental operations manually often leads to challenges such as data inconsistency, booking conflicts, delayed updates, and human errors.

Tracking customer details, vehicle availability, and payments through traditional methods becomes inefficient and time-consuming.

Hence, there is a need for a digital system that can handle all vehicle rental activities systematically and ensure data accuracy and reliability.

Objectives

- To design a structured and efficient relational database that accurately stores and manages information related to customers, vehicles, bookings, payments, billing, and maintenance operations.
- To automate key vehicle rental processes such as booking, billing, payment tracking, and vehicle availability management, reducing manual effort and operational delays.
- To ensure data accuracy, consistency, and integrity through the use of primary keys, foreign keys, constraints, and well-defined relationships between entities.
- To provide a reliable system for quick data retrieval, enabling users to easily access information such as vehicle details, customer history, booking status, and transaction records.
- To improve decision-making and operational clarity by generating essential reports related to revenue, vehicle utilization, maintenance activities, and customer feedback.
- To enhance user experience by offering a structured and organized flow of information that simplifies the overall vehicle rental management process.

1. Requirement Analysis

- Understanding the Purpose of the Project

The aim of the Vehicle Rental Management System is to digitalize the process of renting vehicles to customers, maintaining vehicle details, tracking bookings, generating bills, processing payments, recording feedback, and managing employees and maintenance activities. The system ensures smooth operations for both customers and the rental company.

- Identifying Users and User Types

There are several user types who interact with the system:

1. Customers – Individuals who rent vehicles and provide feedback.
2. Employees – Staff who manage vehicle bookings, maintenance, and operations.
3. Admin/System Manager – Manages vehicle models, types, companies, employee data, and overall system updates.

Understanding the roles of these users helps determine the core entities such as Customer, Employee, Booking, Vehicle, Vehicle Model, etc.

- Analyzing User Interactions with the System

Different users interact with different parts of the system:

- Customers can register, make bookings, return vehicles, make payments, and give feedback.
- Employees handle maintenance tasks and operational activities.
- Admin updates vehicle availability, model details, company details, insurance, etc.

These interactions directly help in identifying the relationships between entities.

For example:

- A customer making a booking ⇒ relationship between Customer and Booking
- A booking generating a bill ⇒ relationship between Booking and Billing

Database Design

Entities of our database

Entity	Key Attributes	Description
customer	cust_id, name, phone_num, mail_id, address fields, license_no	Stores customer personal and contact details.
company	com_id, com_name, com_location	Represents vehicle manufacturing companies.
vehicle_type	type_id, type_name	Categorizes vehicles (SUV, Sedan, etc.).
vehicle_model	vehicle_id, model, seats, ac, lug_cap, type_id, company_id, rent_per_day, penalty	Defines specifications and pricing of each vehicle model.
vehicle	vehicle_num, vehicle_id, reg_no, avail_status	Stores individual vehicle information.
booking	booking_id, cust_id, vehicle_num, start_date/time, end_date/time, pay_amount	Records rental bookings made by customers.

Entity	Key Attributes	Description
billing	bill_id, booking_id, return_date/time, penalty_amount, total_amount	Stores billing details after a booking is completed.
payment	payment_id, billing_id, booking_id, payment_mode, payment_for, amount_paid	Contains information about payments made by customers.
employee	emp_id, emp_name, role, date_of_join, salary	Stores employee details in the rental company.
maintenance	maintenance_id, vehicle_num, service, date/time, expenditure, emp_id	Tracks vehicle service and maintenance activity.
insurance	insurance_id, vehicle_num, provider, expiry_date, policy_description	Stores insurance details for each vehicle.
feedback	feedback_id, booking_id, customer_id, rating, comments	Stores customer reviews and feedback for bookings.

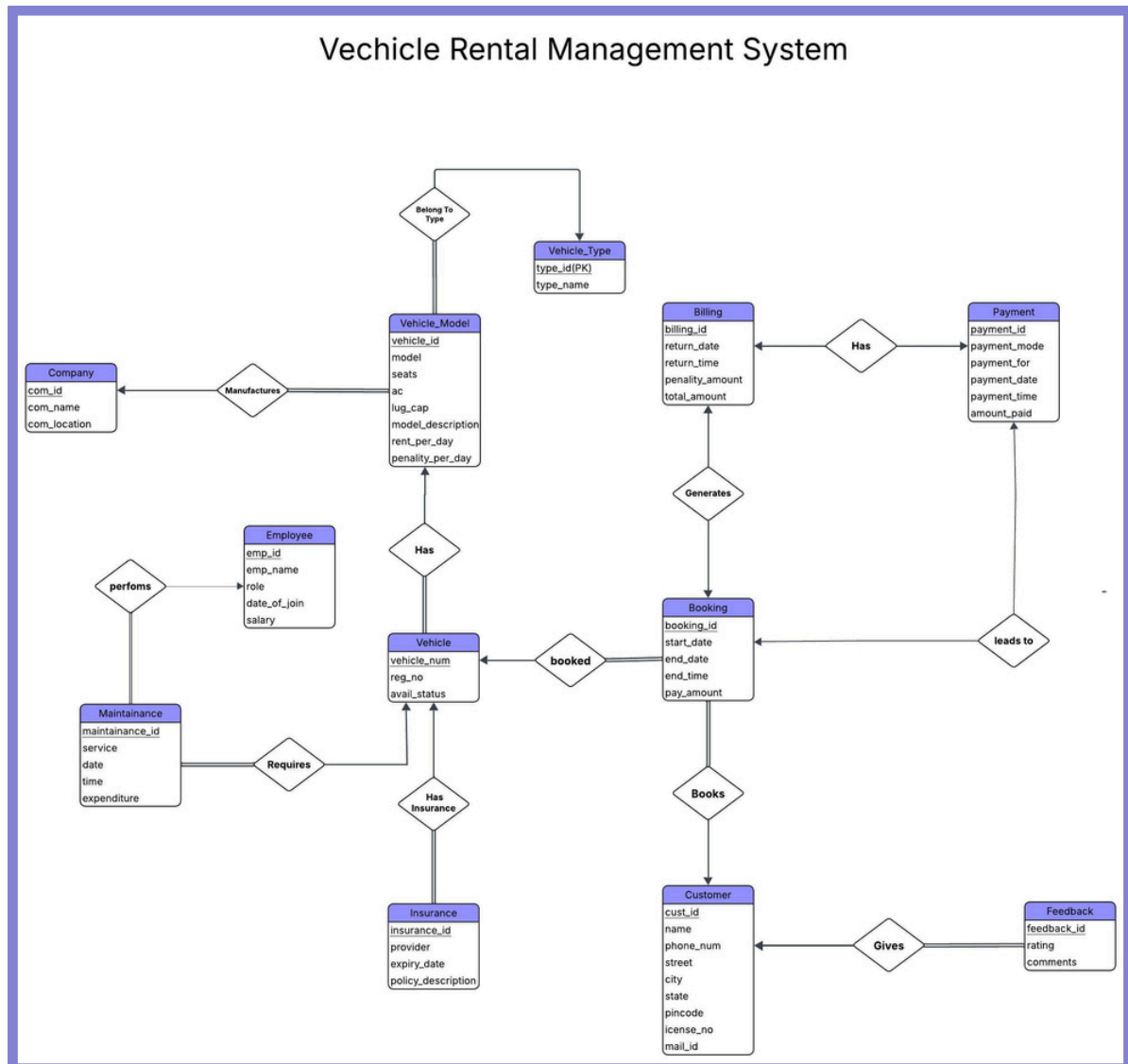
Relationships of our database

The Relationships that are interactions between the entities along with their cardinality and participation are shown below

Relationship	Entities Involved	Cardinality	Participation
Books	Customer ↔ Booking	1 → M	Customer (Partial), Booking (Total)
booked	Vehicle ↔ Booking	1 → M	Vehicle (Partial), Booking (Total)
Generates	Booking ↔ Billing	1 → 1	Both Total
Has	Billing ↔ Payment	1 → 1	Billing (Partial), Payment (Total)
leads to	Booking ↔ Payment	1 → 1	Booking (Partial), Payment (Total)
Manufactures	Company ↔ Vehicle Model	1 → M	Company (Partial), Model (Total)
Belong To Type	Vehicle Type ↔ Vehicle Model	1 → M	Type (Partial), Model (Total)
Has	Vehicle Model ↔ Vehicle	1 → M	Model (Partial), Vehicle (Total)
Requires	Vehicle ↔ Maintenance	1 → M	Vehicle (Partial), Maintenance (Total)
performs	Employee ↔ Maintenance	1 → M	Employee (Partial), Maintenance (Total)
Has Insurance	Vehicle ↔ Insurance	1 → M	Vehicle (Partial), Insurance (Total)
Gives	Customer ↔ Feedback	1 → M	Customer (Partial), Feedback (Total)

2. ER Diagram of the database

Based on the above entities and relationships the following ER Diagram is generated:

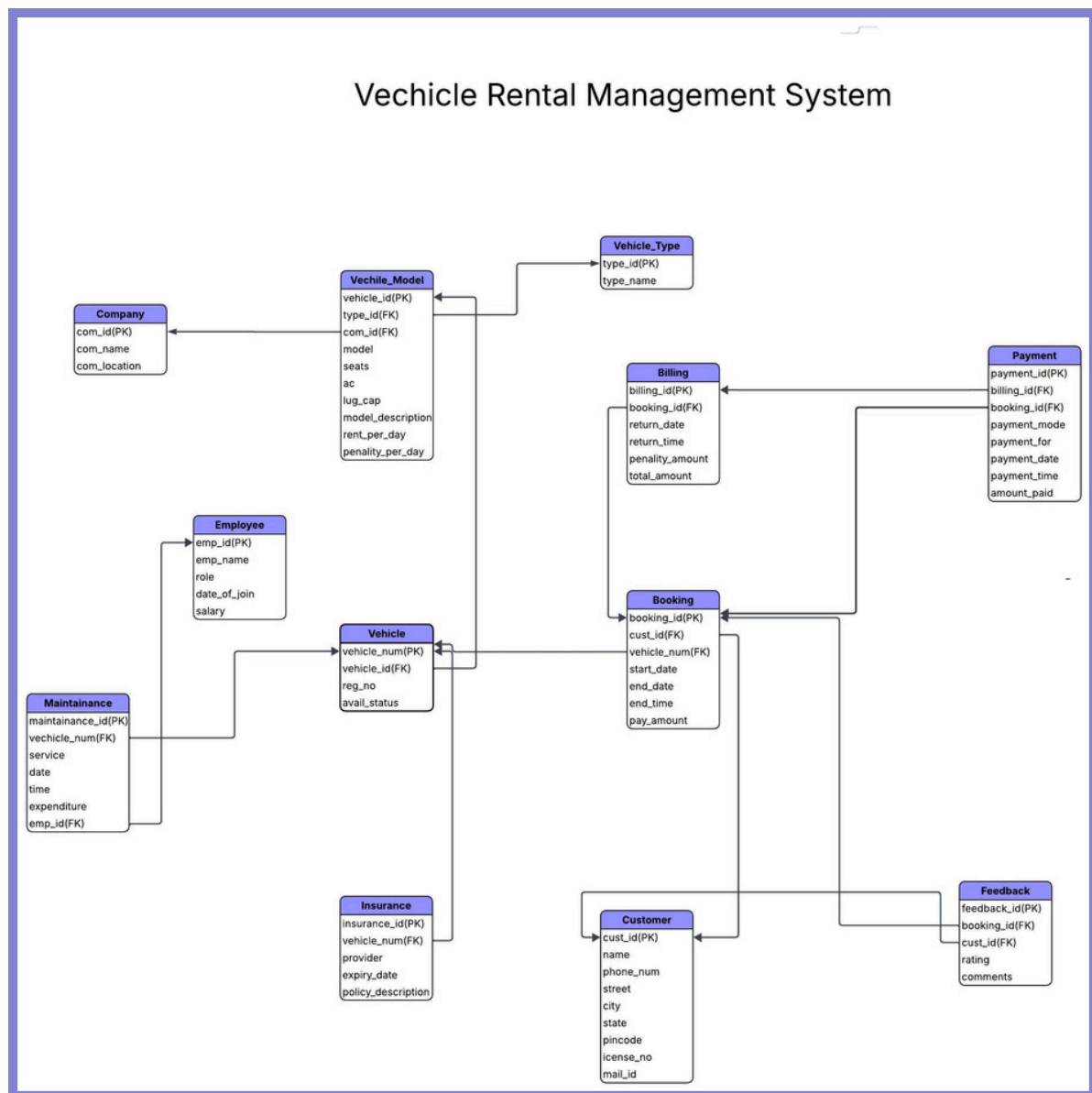


Link for our Design of ER Diagram in lucid chart! : [Link](#)

3. Logical Design (Database Schema)

Relation Schema

The ER Diagram is further transformed to Relational Schema



Link for our Design of ER Diagram in lucid chart! : [Link](#)

SQL Implementation

Create a database and use it:

```
create database Vehicle_Rental_System;  
use Vehicle_Rental_System;
```

Creating the tables

1. customers

```
create table customers (  
    cust_id int primary key auto_increment,  
    name varchar(100) not null,  
    phone_num varchar(20) unique,  
    mail_id varchar(255) unique,  
    street varchar(120),  
    city varchar(80),  
    state varchar(80),  
    pincode varchar(10) check (char_length(pincode) = 6),  
    license_no varchar(50) not null unique  
);
```

2. vehicle_model

```
create table vehicle_model (  
    vehicle_id varchar(20) primary key,  
    model varchar(120) not null,  
    seats int not null check (seats >= 2),  
    ac boolean not null,  
    lug_cap int,  
    type_id int not null,  
    model_description text,  
    company_id int not null,  
    rent_per_day decimal(10,2) not null check (rent_per_day > 0),  
    penalty decimal(10,2) check (penalty >= 0),  
    foreign key (type_id) references vehicle_type(type_id),  
    foreign key (company_id) references company(com_id)  
);
```

3. company

```
create table company (  
  com_id int primary key auto_increment,  
  com_name varchar(120) not null unique,  
  com_location varchar(120)  
);
```

4. vehicle_type

```
create table vehicle_type (  
  type_id int primary key auto_increment,  
  type_name varchar(60) not null unique  
);
```

5. vehicles

```
create table vehicles (  
  vehicle_num varchar(32) primary key,  
  vehicle_id varchar(20) not null,  
  reg_no varchar(64) not null unique,  
  avail_status varchar(20) not null check (avail_status in  
( 'available', 'unavailable', 'in_service' )),  
  foreign key (vehicle_id) references vehicle_model(vehicle_id)  
);
```

6. billing

```
create table billing (  
  bill_id int primary key auto_increment,  
  booking_id int not null unique,  
  return_date date not null,  
  return_time time not null,  
  penalty_amount decimal(12,2) default 0,  
  total_amount decimal(12,2) not null check (total_amount >= 0),  
  foreign key (booking_id) references booking(booking_id)  
);
```

7. booking

```
create table booking (  
    booking_id int primary key auto_increment,  
    cust_id int not null,  
    vehicle_num varchar(32) not null,  
    start_date date not null,  
    start_time time not null,  
    end_date date not null,  
    end_time time not null,  
    pay_amount decimal(12,2),  
    check (end_date >= start_date),  
    foreign key (cust_id) references customers(cust_id),  
    foreign key (vehicle_num) references vehicles(vehicle_num)  
);
```

8. payment

```
create table payment (  
    payment_id int primary key auto_increment,  
    billing_id int,  
    booking_id int not null,  
    payment_mode varchar(40) check (payment_mode in  
( 'cash','card','upi','netbanking','wallet' )),  
    payment_for varchar(60),  
    payment_date date not null,  
    payment_time time not null,  
    amount_paid decimal(12,2) not null check (amount_paid > 0),  
    foreign key (billing_id) references billing(bill_id),  
    foreign key (booking_id) references booking(booking_id)  
);
```

9. employee

```
create table employee (  
    emp_id int primary key auto_increment,  
    emp_name varchar(120) not null,  
    role varchar(80) not null,  
    date_of_join date not null,  
    salary decimal(12,2) check (salary >= 0)  
);
```

10. maintainance

```
create table maintainance (  
    maintainance_id int primary key auto_increment,  
    vehicle_num varchar(32) not null,  
    service varchar(150) not null,  
    service_date date not null,  
    service_time time not null,  
    expenditure decimal(12,2) check (expenditure >= 0),  
    emp_id int not null,  
    foreign key (vehicle_num) references vehicles(vehicle_num),  
    foreign key (emp_id) references employee(emp_id)  
);
```

11. insurance

```
create table insurance (  
    insurance_id int primary key auto_increment,  
    vehicle_num varchar(32) not null,  
    provider varchar(120) not null,  
    expiry_date date not null,  
    policy_description text,  
    foreign key (vehicle_num) references vehicles(vehicle_num)  
);
```

12. feedback

```
create table feedback (  
    feedback_id int primary key auto_increment,  
    booking_id int not null,  
    customer_id int not null,  
    rating int check (rating between 1 and 5),  
    comments text,  
    foreign key (booking_id) references booking(booking_id),  
    foreign key (customer_id) references customers(cust_id)  
);
```

Entity details

Table Name	Primary Key	Foreign Keys (References)	Constraints Applied
customers	cust_id	—	name NOT NULL phone_num UNIQUE mail_id UNIQUE pincode CHECK (length(pincode)=6)
company	com_id	—	com_name UNIQUE com_location NOT NULL
vehicle_type	type_id	—	type_name UNIQUE type_name NOT NULL
vehicle_model	vehicle_id	type_id → vehicle_type(type_id) company_id → company(com_id)	model NOT NULL rent_per_day CHECK (rent_per_day > 0) penalty CHECK (penalty >= 0)
vehicles	vehicle_num	vehicle_id → vehicle_model(vehicle_id)	reg_no UNIQUE avail_status CHECK (avail_status IN ('available','unavailable','in_service'))
booking	booking_id	cust_id → customers(cust_id) vehicle_num → vehicles(vehicle_num)	pay_amount CHECK (pay_amount >= 0) start_date < end_date

Entity details

Table Name	Primary Key	Foreign Keys (References)	Constraints Applied
billing	bill_id	booking_id → booking(booking_id)	penalty_amount CHECK (penalty_amount >= 0) total_amount CHECK (total_amount >= 0)
payment	payment_id	billing_id → billing(bill_id) booking_id → booking(booking_id)	amount_paid CHECK (amount_paid >= 0) payment_mode NOT NULL
employee	emp_id	—	emp_name NOT NULL salary CHECK (salary > 0) role NOT NULL
maintainance	maintainance_id	vehicle_num → vehicles(vehicle_num) emp_id → employee(emp_id)	expenditure CHECK (expenditure >= 0) service NOT NULL
insurance	insurance_id	vehicle_num → vehicles(vehicle_num)	provider NOT NULL, expiry_date > CURDATE() policy_description NOT NULL
feedback	feedback_id	booking_id → booking(booking_id) customer_id → customers(cust_id)	rating CHECK (rating BETWEEN 1 AND 5)

4. Normalization

Step 1 :

Starting relation (1NF)

We start from a single denormalized relation that contains all attributes listed:

```
FullVehicle(  
  com_id,  
  com_name,  
  com_location,  
  type_id,  
  type_name,  
  vehicle_id,  
  model,  
  seats,  
  ac,  
  lug_cap,  
  rent_per_day,  
  penalty,  
  vehicle_num,  
  reg_no,  
  avail_status,  
  emp_id,  
  emp_name,  
  role,  
  date_of_join,  
  salary,  
  maintainance_id,  
  service,  
  date/time,  
  expenditure,  
  insurance_id,  
  provider,  
  expiry_date,  
  policy_description  
)
```

4. Normalization

1NF check

Rule: 1NF requires all attributes be atomic (no repeating groups, no multivalued or composite attributes).

Check: Every attribute in the list is atomic (IDs, names, numeric, date/time, text). $\Rightarrow$ 1NF satisfied.

Functional dependencies (FDs)

- $\text{com_id} \rightarrow \text{com_name}, \text{com_location}$
- $\text{type_id} \rightarrow \text{type_name}$
- $\text{vehicle_id} \rightarrow \text{model}, \text{seats}, \text{ac}, \text{lug_cap}, \text{type_id}, \text{com_id}, \text{rent_per_day}, \text{penalty}$
- $\text{vehicle_num} \rightarrow \text{vehicle_id}, \text{reg_no}, \text{avail_status}$
- $\text{emp_id} \rightarrow \text{emp_name}, \text{role}, \text{date_of_join}, \text{salary}$
- $\text{maintainance_id} \rightarrow \text{vehicle_num}, \text{service}, \text{date/time}, \text{expenditure}, \text{emp_id}$
- $\text{insurance_id} \rightarrow \text{vehicle_num}, \text{provider}, \text{expiry_date}, \text{policy_description}$

Candidate keys (CKs)

State these CKs (PK choices) exactly:

- Company: com_id (PK)
- VehicleType: type_id (PK)
- VehicleModel: vehicle_id (PK)
- Vehicle (instance): vehicle_num (PK)
- Employee: emp_id (PK)
- Maintenance: maintainance_id (PK)
- Insurance: insurance_id (PK)

Step 2 :

Checking 2NF

2NF Rule :

A relation is in 2NF if it is in 1NF and no non-prime attribute is partially dependent on a proper subset of any candidate key.

For each FD $X \rightarrow A$, check whether X is a proper subset of any composite CK. If so and A is non-prime, a partial dependency exists and 2NF is violated.

4. Normalization

Application to our FullVehicle:

The denormalized relation mixes multiple entity types (company, model, type, vehicle instance, employee, maintenance, insurance). Non-prime attributes such as model, seats, ac, lug_cap, rent_per_day, penalty depend on vehicle_id (model) not on vehicle_num. Similarly com_name, com_location depend on com_id, and type_name depends on type_id. These are partial dependencies relative to a vehicle-row view.

Conclusion: FullVehicle is not in 2NF because model/type/company attributes partially depend on smaller identifiers (vehicle_id, type_id, com_id) rather than on the full row key. Decomposition is required.

Decomposition steps to reach 2NF :

Perform these steps (numbered) — these produce the normalized relations using your exact names:

- Create Company(com_id PK, com_name, com_location).
- Create VehicleType(type_id PK, type_name).
- Create VehicleModel(vehicle_id PK, model, seats, ac, lug_cap, type_id FK → VehicleType, com_id FK , rent_per_day, penalty).
- Create Vehicle(vehicle_num PK, vehicle_id FK , reg_no, avail_status).
- Create Employee(emp_id PK, emp_name, role, date_of_join, salary).
- Create Maintenance(maintenance_id PK, vehicle_num FK , service, date/time, expenditure, emp_id FK).
- Create Insurance(insurance_id PK, vehicle_num FK , provider, expiry_date, policy_description).

After these steps: model/type/company attributes are stored in their own relations and no non-prime attribute is partially dependent on a subset of a composite key. $\Rightarrow$ 2NF achieved.

Step 3 :

Checking 3NF

3NF Rule :

A relation is in 3NF if it is in 2NF and for every FD $X \rightarrow A$ either X is a superkey or A is a prime attribute (part of some candidate key).

4. Normalization

How to check:

For each decomposed relation list applicable FDs.

For each FD $X \rightarrow Y$, verify X is a superkey of that relation or Y is prime. If neither holds, transitive dependency exists and further decomposition is required.

Application to current relations:

- Company: $\text{com_id} \rightarrow \text{com_name}, \text{com_location}$ — com_id is PK (superkey) $\Rightarrow$ OK.
- VehicleType: $\text{type_id} \rightarrow \text{type_name}$ — type_id is PK $\Rightarrow$ OK.
- VehicleModel: $\text{vehicle_id} \rightarrow \text{model}, \text{seats}, \text{ac}, \text{lug_cap}, \text{type_id}, \text{com_id}, \text{rent_per_day}, \text{penalty}$ — vehicle_id is PK $\Rightarrow$ OK. type_id and com_id are FKs but not leading to transitive dependencies inside this relation.
- Vehicle: $\text{vehicle_num} \rightarrow \text{vehicle_id}, \text{reg_no}, \text{avail_status}$ — vehicle_num is PK $\Rightarrow$ OK.
- Employee: $\text{emp_id} \rightarrow \text{emp_name}, \text{role}, \text{date_of_join}, \text{salary}$ — emp_id is PK $\Rightarrow$ OK.
- Maintenance: $\text{maintainance_id} \rightarrow \text{vehicle_num}, \text{service}, \text{date/time}, \text{expenditure}, \text{emp_id}$ — PK LHS $\Rightarrow$ OK.
- Insurance: $\text{insurance_id} \rightarrow \text{vehicle_num}, \text{provider}, \text{expiry_date}, \text{policy_description}$ — PK LHS $\Rightarrow$ OK.

Transitive dependency check: the original transitive chain $\text{vehicle_num} \rightarrow \text{vehicle_id}$ and $\text{vehicle_id} \rightarrow \text{com_id}$ and $\text{com_id} \rightarrow \text{com_name}$ would have implied $\text{vehicle_num} \rightarrow \text{com_name}$ transitively. After decomposition com_name is only in Company, so no transitive dependency remains inside a single relation.

Conclusion: All decomposed relations satisfy 3NF. Transitive dependencies have been eliminated.

Step 4:

BCNF check (stricter form)

BCNF Rule (to include):

A relation is in BCNF if for every non-trivial FD $X \rightarrow Y$, X is a superkey.

Check: In each final relation above the non-trivial FDs have LHS equal to the declared PK (com_id , type_id , vehicle_id , vehicle_num , emp_id , maintainance_id , insurance_id). Therefore each relation satisfies BCNF under the discovered FDs.

4. Normalization

Final normalized relations

- Company
Attributes: com_id (PK), com_name, com_location
- VehicleType
Attributes: type_id (PK), type_name
- VehicleModel
Attributes: vehicle_id (PK), model, seats, ac, lug_cap, type_id , com_id, rent_per_day, penalty
- Vehicle
Attributes: vehicle_num (PK), vehicle_id , reg_no, avail_status
- Employee
Attributes: emp_id (PK), emp_name, role, date_of_join, salary
- Maintenance
Attributes: maintainance_id (PK), vehicle_num , service, date/time, expenditure, emp_id
- Insurance
Attributes: insurance_id (PK), vehicle_num (FK → Vehicle), provider, expiry_date, policy_description
(These relations are in 3NF and satisfy BCNF under the stated FDs.)

Final conclusion

Starting from the denormalized FullVehicle relation, we identified the functional dependencies and candidate keys, removed partial dependencies to reach 2NF by creating Company, VehicleType, and VehicleModel tables, and eliminated transitive dependencies to reach 3NF (and BCNF under the given FDs). The final relations (Company, VehicleType, VehicleModel, Vehicle, Employee, Maintenance, Insurance) minimize redundancy and prevent insertion/update/delete anomalies while preserving all original information via foreign keys.

5. Data Population

The following are the screenshots of execution of creation of database and tables in MySQL and also insertion of data into tables. It is ensured that data of 15 to 20 tuples (records) is maintained in the tables to ensure efficient working of queries.

Creating database

```
MySQL localhost:33060+ ssl SQL > create database Vehicle_Rental_System;
Query OK, 1 row affected (0.0042 sec)
MySQL localhost:33060+ ssl SQL > use Vehicle_Rental_System;
Default schema set to 'Vehicle_Rental_System'.
Fetching global names, object names from 'vehicle_rental_system' for auto-completion... Press ^C to stop.
MySQL localhost:33060+ ssl vehicle_rental_system SQL > |
```

Creating tables and inserting data into it

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table customers (
-> cust_id int primary key auto_increment,
-> name varchar(100) not null,
-> phone_num varchar(20) unique,
-> mail_id varchar(255) unique,
-> street varchar(120),
-> city varchar(80),
-> state varchar(80),
-> pincode varchar(10) check (char_length(pincode) = 6),
-> license_no varchar(50) not null unique
-> );
Query OK, 0 rows affected (0.0198 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into customers (name, phone_num, mail_id, street, city, state, pincode, license_no) values
-> ('arjun reddy', '9876543210', 'arjun.reddy@gmail.com', '12-4-56', 'hyderabad', 'telangana', '500001', 'TS09L12345'),
-> ('priya sharma', '9876543211', 'priya.sharma@gmail.com', '22-1-45', 'mumbai', 'maharashtra', '400001', 'MH01L23456'),
-> ('rahul kumar', '9876543212', 'rahul.kumar@gmail.com', '5-2-33', 'delhi', 'delhi', '110001', 'DL04L98765'),
-> ('ananya verma', '9876543213', 'ananya.verma@gmail.com', '44-3-12', 'kolkata', 'west bengal', '700001', 'WB20L34891'),
-> ('sai kiran', '9876543214', 'sai.kiran@gmail.com', '18-7-22', 'vijayawada', 'andhra pradesh', '520001', 'AP07L12987'),
-> ('isha patel', '9876543215', 'isha.patel@gmail.com', '9-6-17', 'ahmedabad', 'gujarat', '380001', 'GJ01L99882'),
-> ('vinit mehta', '9876543216', 'vinit.mehta@gmail.com', '6-5-41', 'surat', 'gujarat', '395003', 'GJ05L66782'),
-> ('deeksha reddy', '9876543217', 'deeksha.reddy@gmail.com', '14-9-22', 'chennai', 'tamil nadu', '600001', 'TN10L88123'),
-> ('karan singh', '9876543218', 'karan.singh@gmail.com', '3-4-17', 'jaipur', 'rajasthan', '302001', 'RJ14L77834'),
-> ('madhu priya', '9876543219', 'madhu.priya@gmail.com', '10-2-5', 'kochi', 'kerala', '682001', 'KL07L44219'),
-> ('sachin naik', '9876543220', 'sachin.naik@gmail.com', '5-2-9', 'pune', 'maharashtra', '411001', 'MH12L92411'),
-> ('revanth varma', '9876543221', 'revanth.varma@gmail.com', '23-5-44', 'warangal', 'telangana', '506001', 'TS03L98211'),
-> ('varsha gowda', '9876543222', 'varsha.gowda@gmail.com', '9-8-12', 'bengaluru', 'karnataka', '560001', 'KA05L78123'),
-> ('rahul sharma', '9876543223', 'rahul.sharma@gmail.com', '11-9-34', 'lucknow', 'uttar pradesh', '226001', 'UP32L99811'),
-> ('meghana joshi', '9876543224', 'meghana.joshi@gmail.com', '2-4-16', 'indore', 'madhya pradesh', '452001', 'MP09L66542'),
-> ('aditya jain', '9876543225', 'aditya.jain@gmail.com', '1-3-14', 'bhopal', 'madhya pradesh', '462001', 'MP04L44122'),
-> ('soumya reddy', '9876543226', 'soumya.reddy@gmail.com', '14-3-51', 'nellore', 'andhra pradesh', '524001', 'AP26L88721'),
-> ('kavya menon', '9876543227', 'kavya.menon@gmail.com', '21-6-33', 'trivandrum', 'kerala', '695001', 'KL01L98213'),
-> ('naveen ch', '9876543228', 'naveen.ch@gmail.com', '8-9-11', 'vizag', 'andhra pradesh', '530001', 'AP31L77139'),
-> ('sanju yadav', '9876543229', 'sanju.yadav@gmail.com', '4-7-19', 'patna', 'bihar', '800001', 'BR01L73542');
Query OK, 20 rows affected (0.0137 sec)
Records: 20 Duplicates: 0 Warnings: 0
```

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table company (
-> com_id int primary key auto_increment,
-> com_name varchar(120) not null unique,
-> com_location varchar(120)
-> );
Query OK, 0 rows affected (0.0184 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into company (com_name, com_location) values
-> ('maruti suzuki', 'manesar'),
-> ('honda motorcycles', 'gurugram'),
-> ('tvs motor', 'chennai'),
-> ('bajaj auto', 'pune'),
-> ('mahindra', 'mumbai'),
-> ('hyundai', 'chennai'),
-> ('tata motors', 'pune'),
-> ('kia motors', 'chennai'),
-> ('toyota', 'bangalore'),
-> ('honda cars', 'gurgaon'),
-> ('royal enfield', 'chennai'),
-> ('hero motocorp', 'gurgaon'),
-> ('piaggio india', 'delhi'),
-> ('force motors', 'pune'),
-> ('ashok leyland', 'chennai'),
-> ('olectra', 'varanasi'),
-> ('ola electric', 'bengaluru'),
-> ('mahindra electric', 'bengaluru'),
-> ('mg motors', 'gurugram'),
-> ('skoda india', 'pune');
Query OK, 20 rows affected (0.0033 sec)
Records: 20 Duplicates: 0 Warnings: 0
```

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table vehicle_type (
->     type_id int primary key auto_increment,
->     type_name varchar(60) not null unique
-> );
```

Query OK, 0 rows affected (0.0159 sec)

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL >
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into vehicle_type (type_name) values
```

```

-> ('car'),
-> ('bike'),
-> ('suv'),
-> ('hatchback'),
-> ('sedan'),
-> ('van'),
-> ('minivan'),
-> ('tempo'),
-> ('bus'),
-> ('truck'),
-> ('electric bike'),
-> ('electric car'),
-> ('luxury car'),
-> ('pickup'),
-> ('crossover'),
-> ('micro'),
-> ('convertible'),
-> ('coupe'),
-> ('three-wheeler'),
-> ('tempo traveller');
```

Query OK, 20 rows affected (0.0024 sec)

Records: 20 Duplicates: 0 Warnings: 0

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table vehicle_model (
->     vehicle_id varchar(20) primary key,
->     model varchar(120) not null,
->     seats int not null check (seats >= 2),
->     ac boolean not null,
->     lug_cap int,
->     type_id int not null,
->     model_description text,
->     company_id int not null,
->     rent_per_day decimal(10,2) not null check (rent_per_day > 0),
->     penalty decimal(10,2) check (penalty >= 0),
->     foreign key (type_id) references vehicle_type(type_id),
->     foreign key (company_id) references company(company_id)
-> );
```

Query OK, 0 rows affected (0.0265 sec)

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into vehicle_model (vehicle_id, model, seats, ac, lug_cap, type_id, model_description, company_id, rent_per_day, penalty) values
-> ('vm001','swift dzire',5, true, 350, 5, 'maruti dzire vxi petrol', 1, 1800.00, 500.00),
-> ('vm002','activa 6g',2, false, 20, 2, 'honda activa 6g 118cc', 2, 400.00, 100.00),
-> ('vm003','pulsar 150',2, false, 18, 2, 'bajaj pulsar 150 ns', 4, 450.00, 120.00),
-> ('vm004','innova crysta',7, true, 500, 3, 'toyota innova crysta 2.4', 9, 3500.00, 800.00),
-> ('vm005','creta sx',5, true, 400, 3, 'hyundai creta sx petrol', 6, 3000.00, 700.00),
-> ('vm006','royal enfield classic 350',2, false, 30, 2, 'classic 350 single cylinder', 11, 700.00, 200.00),
-> ('vm007','eeco 7 seater',7, true, 600, 6, 'maruti eeco 7 seater petrol', 1, 2200.00, 500.00),
-> ('vm008','seltos',5, true, 450, 3, 'kia seltos top variant', 8, 3200.00, 700.00),
-> ('vm009','alto 800',4, true, 200, 4, 'maruti alto 800 standard', 1, 900.00, 250.00),
-> ('vm010','gli sedan',5, true, 400, 5, 'toyota etios gd', 9, 1500.00, 400.00),
-> ('vm011','electric scooter sl',2, false, 15, 11, 'ola sl pro electric', 17, 500.00, 150.00),
-> ('vm012','tata nexon ev',5, true, 350, 12, 'tata nexon ev', 7, 3800.00, 900.00),
-> ('vm013','harley style',2, false, 25, 2, 'royal enfield style cruiser', 11, 1200.00, 300.00),
-> ('vm014','tempo traveller 12 seater',12, true, 1200, 20, 'force tempo traveller', 14, 4200.00, 1000.00),
-> ('vm015','mahindra bolero',7, true, 500, 6, 'mahindra bolero maxi', 5, 2000.00, 500.00),
-> ('vm016','pulsar ns200',2, false, 25, 2, 'bajaj pulsar ns200', 4, 450.00, 120.00),
-> ('vm017','creta sport edition',5, true, 420, 3, 'hyundai creta sport', 6, 3400.00, 750.00),
-> ('vm018','etios livra',5, true, 300, 4, 'toyota etios livra', 9, 1100.00, 300.00),
-> ('vm019','eeco cargo van',2, false, 800, 6, 'maruti eeco cargo', 1, 1600.00, 400.00),
-> ('vm020','mini truck',2, false, 1000, 10, 'small cargo pickup', 10, 2500.00, 600.00);
```

Query OK, 20 rows affected (0.0050 sec)

Records: 20 Duplicates: 0 Warnings: 0

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table vehicles (
->     vehicle_num varchar(32) primary key,
->     vehicle_id varchar(20) not null,
->     reg_no varchar(64) not null unique,
->     avail_status varchar(20) not null check (avail_status in ('available','unavailable','in_service')),
->     foreign key (vehicle_id) references vehicle_model(vehicle_id)
-> );
```

Query OK, 0 rows affected (0.0166 sec)

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into vehicles (vehicle_num, vehicle_id, reg_no, avail_status) values
-> ('AP001','vm001','mh12ab1234','available'),
-> ('AP002','vm002','ts09cd4321','available'),
-> ('AP003','vm003','ka05ef6789','unavailable'),
-> ('AP004','vm004','dl04gh1111','in_service'),
-> ('AP005','vm005','ap31jk2222','available'),
-> ('AP006','vm006','tn10lm3333','available'),
-> ('AP007','vm007','wb20no4444','available'),
-> ('AP008','vm008','hr26pq5555','unavailable'),
-> ('AP009','vm009','mh14rs6666','available'),
-> ('AP010','vm010','up32tu7777','available'),
-> ('AP011','vm011','ka03uv8888','available'),
-> ('AP012','vm012','dl05wx9999','available'),
-> ('AP013','vm013','ga01yz1010','available'),
-> ('AP014','vm014','pb02aa2020','available'),
-> ('AP015','vm015','mp09bb3030','in_service'),
-> ('AP016','vm016','mh20cc0000','available'),
-> ('AP017','vm017','tn07dd5050','available'),
-> ('AP018','vm018','kl07ee6060','available'),
-> ('AP019','vm019','gj05ff7070','available'),
-> ('AP020','vm020','rjl4gg8080','available');
```

Query OK, 20 rows affected (0.0031 sec)

Records: 20 Duplicates: 0 Warnings: 0


```

MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table booking (
-> booking_id int primary key auto_increment,
-> cust_id int not null,
-> vehicle_num varchar(32) not null,
-> start_date date not null,
-> start_time time not null,
-> end_date date not null,
-> end_time time not null,
-> pay_amount decimal(12,2),
-> check (end_date >= start_date),
-> foreign key (cust_id) references customers(cust_id),
-> foreign key (vehicle_num) references vehicles(vehicle_num)
-> );

Query OK, 0 rows affected (0.0179 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into booking (cust_id, vehicle_num, start_date, start_time, end_date, end_time, pay_amount) values
-> (1,'AP001','2025-10-20','09:00:00','2025-10-23','09:00:00',5400.00),
-> (2,'AP002','2025-10-22','10:30:00','2025-10-24','10:30:00',800.00),
-> (3,'AP003','2025-10-25','08:00:00','2025-10-27','18:00:00',900.00),
-> (4,'AP004','2025-10-28','09:30:00','2025-10-30','09:30:00',7000.00),
-> (5,'AP005','2025-10-29','11:00:00','2025-10-31','11:00:00',6000.00),
-> (6,'AP006','2025-10-30','07:00:00','2025-11-02','07:00:00',2100.00),
-> (7,'AP007','2025-11-01','10:00:00','2025-11-03','10:00:00',4400.00),
-> (8,'AP008','2025-11-02','14:00:00','2025-11-04','14:00:00',9600.00),
-> (9,'AP009','2025-11-03','09:00:00','2025-11-05','09:00:00',2700.00),
-> (10,'AP010','2025-10-18','12:00:00','2025-10-20','12:00:00',3000.00),
-> (11,'AP011','2025-10-21','08:30:00','2025-10-22','08:30:00',500.00),
-> (12,'AP012','2025-10-23','09:15:00','2025-10-25','09:15:00',7600.00),
-> (13,'AP013','2025-10-24','07:45:00','2025-10-26','07:45:00',2400.00),
-> (14,'AP014','2025-10-26','06:30:00','2025-10-28','06:30:00',12600.00),
-> (15,'AP015','2025-10-27','10:00:00','2025-10-29','10:00:00',4000.00),
-> (16,'AP016','2025-10-28','11:20:00','2025-10-30','11:20:00',1100.00),
-> (17,'AP017','2025-10-29','09:40:00','2025-10-31','09:40:00',10200.00),
-> (18,'AP018','2025-10-30','08:10:00','2025-11-01','08:10:00',3300.00),
-> (19,'AP019','2025-10-31','07:50:00','2025-11-02','07:50:00',3200.00),
-> (20,'AP020','2025-11-01','13:00:00','2025-11-03','13:00:00',5000.00);

Query OK, 20 rows affected (0.0049 sec)

Records: 20 Duplicates: 0 Warnings: 0

```

```

MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table billing (
-> bill_id int primary key auto_increment,
-> booking_id int not null unique,
-> return_date date not null,
-> return_time time not null,
-> penalty_amount decimal(12,2) default 0,
-> total_amount decimal(12,2) not null check (total_amount >= 0),
-> foreign key (booking_id) references booking(booking_id)
-> );

Query OK, 0 rows affected (0.0143 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into billing (booking_id, return_date, return_time, penalty_amount, total_amount) values
-> (1,'2025-10-25','10:00:00',1000.00,6400.00),
-> (2,'2025-10-26','11:00:00',200.00,1000.00),
-> (3,'2025-10-29','09:00:00',360.00,1260.00),
-> (4,'2025-11-02','08:00:00',1600.00,8600.00),
-> (5,'2025-11-03','12:00:00',1400.00,7400.00),
-> (6,'2025-11-05','10:00:00',600.00,2700.00),
-> (7,'2025-11-06','12:30:00',500.00,4900.00),
-> (8,'2025-11-07','15:00:00',1400.00,11000.00),
-> (9,'2025-11-07','10:00:00',500.00,3200.00),
-> (10,'2025-10-23','13:00:00',800.00,3800.00),
-> (11,'2025-10-24','09:00:00',150.00,650.00),
-> (12,'2025-10-28','10:00:00',1800.00,9400.00),
-> (13,'2025-10-29','08:00:00',600.00,3000.00),
-> (14,'2025-10-31','07:00:00',2000.00,14600.00),
-> (15,'2025-11-01','11:00:00',1000.00,5000.00),
-> (16,'2025-11-01','12:00:00',240.00,1340.00),
-> (17,'2025-11-02','09:00:00',1500.00,11700.00),
-> (18,'2025-11-03','08:30:00',900.00,4200.00),
-> (19,'2025-11-04','09:20:00',800.00,4000.00),
-> (20,'2025-11-05','14:30:00',600.00,5600.00);

Query OK, 20 rows affected (0.0030 sec)

Records: 20 Duplicates: 0 Warnings: 0

```

```

MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table payment (
->     payment_id int primary key auto_increment,
->     billing_id int,
->     booking_id int not null,
->     payment_mode varchar(40) check (payment_mode in ('cash','card','upi','netbanking','wallet')),
->     payment_for varchar(60),
->     payment_date date not null,
->     payment_time time not null,
->     amount_paid decimal(12,2) not null check (amount_paid > 0),
->     foreign key (billing_id) references billing(billing_id),
->     foreign key (booking_id) references booking(booking_id)
-> );

Query OK, 0 rows affected (0.0191 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into payment (billing_id, booking_id, payment_mode, payment_for, payment_date, payment_time, amount_paid) values
-> (null,1,'card','actual','2025-10-20','09:05:00',5400.00),
-> (null,2,'upi','actual','2025-10-22','10:35:00',800.00),
-> (null,3,'cash','actual','2025-10-25','08:10:00',900.00),
-> (null,4,'netbanking','actual','2025-10-28','09:35:00',7000.00),
-> (null,5,'card','actual','2025-10-29','11:05:00',6000.00),
-> (null,6,'upi','actual','2025-10-30','07:10:00',2100.00),
-> (null,7,'cash','actual','2025-11-01','10:05:00',4400.00),
-> (null,8,'card','actual','2025-11-02','14:10:00',9600.00),
-> (null,9,'upi','actual','2025-11-03','09:05:00',2700.00),
-> (null,10,'card','actual','2025-10-18','12:05:00',3000.00),
-> (null,11,'cash','actual','2025-10-21','08:35:00',500.00),
-> (null,12,'upi','actual','2025-10-23','09:20:00',7600.00),
-> (null,13,'card','actual','2025-10-24','07:50:00',2400.00),
-> (null,14,'netbanking','actual','2025-10-26','06:35:00',12600.00),
-> (null,15,'card','actual','2025-10-27','10:05:00',4000.00),
-> (null,16,'upi','actual','2025-10-28','11:25:00',1100.00),
-> (null,17,'card','actual','2025-10-29','09:45:00',10200.00),
-> (null,18,'cash','actual','2025-10-30','08:15:00',3300.00),
-> (null,19,'upi','actual','2025-10-31','07:55:00',3200.00),
-> (null,20,'card','actual','2025-11-01','13:05:00',5000.00);

Query OK, 20 rows affected (0.0037 sec)

Records: 20 Duplicates: 0 Warnings: 0

```

```

MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into payment (billing_id, booking_id, payment_mode, payment_for, payment_date, payment_time, amount_paid) values
-> (1,1,'card','penalty','2025-10-25','10:05:00',1000.00),
-> (2,2,'upi','penalty','2025-10-26','11:05:00',200.00),
-> (3,3,'cash','penalty','2025-10-29','09:05:00',360.00),
-> (4,4,'netbanking','penalty','2025-11-02','08:05:00',1600.00),
-> (5,5,'card','penalty','2025-11-03','12:10:00',1400.00),
-> (6,6,'upi','penalty','2025-11-05','10:05:00',600.00),
-> (7,7,'cash','penalty','2025-11-06','12:35:00',500.00),
-> (8,8,'card','penalty','2025-11-07','15:05:00',1400.00),
-> (9,9,'upi','penalty','2025-11-07','10:05:00',500.00),
-> (10,10,'card','penalty','2025-10-23','13:05:00',800.00),
-> (11,11,'cash','penalty','2025-10-24','09:05:00',150.00),
-> (12,12,'upi','penalty','2025-10-28','10:05:00',1800.00),
-> (13,13,'card','penalty','2025-10-29','08:05:00',600.00),
-> (14,14,'netbanking','penalty','2025-10-31','07:05:00',2000.00),
-> (15,15,'card','penalty','2025-11-01','11:05:00',1000.00),
-> (16,16,'upi','penalty','2025-11-01','12:05:00',240.00),
-> (17,17,'card','penalty','2025-11-02','09:05:00',1500.00),
-> (18,18,'cash','penalty','2025-11-03','08:35:00',900.00),
-> (19,19,'upi','penalty','2025-11-04','09:25:00',800.00),
-> (20,20,'card','penalty','2025-11-05','14:35:00',600.00);

Query OK, 20 rows affected (0.0047 sec)

Records: 20 Duplicates: 0 Warnings: 0

```

```

MySQL localhost:33060+ ssl vehicle_rental_system SQL > create table employee (
->     emp_id int primary key auto_increment,
->     emp_name varchar(120) not null,
->     role varchar(80) not null,
->     date_of_join date not null,
->     salary decimal(12,2) check (salary >= 0)
-> );

Query OK, 0 rows affected (0.0140 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into employee (emp_name, role, date_of_join, salary) values
-> ('rahul deshमुख','manager','2023-01-10',45000.00),
-> ('sneha rao','service-engineer','2022-05-12',30000.00),
-> ('amit sharma','agent','2024-02-01',25000.00),
-> ('meena kumari','accountant','2021-11-15',32000.00),
-> ('vikas gupta','mechanic','2019-07-20',28000.00),
-> ('pranam singh','cleaner','2020-03-05',15000.00),
-> ('neha patel','customer-exec','2023-08-01',24000.00),
-> ('arvind nair','operations','2022-12-10',35000.00),
-> ('kajal menon','hr','2020-06-18',33000.00),

-> ('suresh r','driver','2018-09-25',22000.00),
-> ('rajiv verma','maintenance-head','2017-04-02',38000.00),
-> ('latha devi','admin','2021-01-07',30000.00),
-> ('manish jain','sales','2023-03-11',26000.00),
-> ('preeti joshi','reception','2024-01-20',21000.00),
-> ('sunil kumar','security','2019-10-10',18000.00),
-> ('poonam k','cleaner','2022-08-30',15000.00),
-> ('aditya r','mechanic','2020-05-21',29000.00),
-> ('divya sharma','data-entry','2023-06-15',23000.00),
-> ('kiran reddy','fleet-manager','2018-02-28',42000.00),
-> ('isha singh','finance','2019-12-19',34000.00);

Query OK, 20 rows affected (0.0054 sec)

Records: 20 Duplicates: 0 Warnings: 0

```

```

MySQL localhost:33060+ ssl vehicle_rental_system SQL> create table maintainance (
-> maintainance_id int primary key auto_increment,
-> vehicle_num varchar(32) not null,
-> service varchar(150) not null,
-> service_date date not null,
-> service_time time not null,
-> expenditure decimal(12,2) check (expenditure >= 0),
-> emp_id int not null,
-> foreign key (vehicle_num) references vehicles(vehicle_num),
-> foreign key (emp_id) references employee(emp_id)
-> );

Query OK, 0 rows affected (0.0089 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL> insert into maintainance (vehicle_num, service, service_date, service_time, expenditure, emp_id) values
-> ('AP001','oil change','2025-09-10','10:00:00',1200.00,5),
-> ('AP002','brake check','2025-09-12','11:30:00',800.00,2),
-> ('AP003','tyre replacement','2025-09-15','09:00:00',4000.00,5),
-> ('AP004','engine service','2025-09-20','08:30:00',8500.00,11),
-> ('AP005','ac repair','2025-09-22','10:15:00',3000.00,17),
-> ('AP006','battery change','2025-09-25','14:00:00',6000.00,16),
-> ('AP007','interior cleaning','2025-09-28','09:45:00',1200.00,6),
-> ('AP008','gearbox repair','2025-09-30','10:30:00',9000.00,5),
-> ('AP009','tyre rotation','2025-10-02','11:00:00',600.00,2),
-> ('AP010','wheel alignment','2025-10-05','09:20:00',700.00,5),
-> ('AP011','software update','2025-10-08','12:00:00',500.00,18),
-> ('AP012','full service','2025-10-10','08:00:00',5500.00,11),
-> ('AP013','chain lubrication','2025-10-12','09:10:00',300.00,2),
-> ('AP014','ac gas refill','2025-10-14','10:40:00',1500.00,8),
-> ('AP015','suspension check','2025-10-16','11:25:00',1200.00,17),
-> ('AP016','spark plug change','2025-10-18','08:50:00',400.00,5),
-> ('AP017','filter replacement','2025-10-20','09:30:00',900.00,5),
-> ('AP018','battery check','2025-10-22','10:10:00',350.00,16),
-> ('AP019','cargo door repair','2025-10-24','11:40:00',2200.00,11),
-> ('AP020','coolant refill','2025-10-26','09:00:00',800.00,5);

Query OK, 20 rows affected (0.0046 sec)

Records: 20 Duplicates: 0 Warnings: 0

MySQL localhost:33060+ ssl vehicle_rental_system SQL> create table insurance (
-> insurance_id int primary key auto_increment,
-> vehicle_num varchar(32) not null,
-> provider varchar(120) not null,
-> expiry_date date not null,
-> policy_description text,
-> foreign key (vehicle_num) references vehicles(vehicle_num)
-> );

Query OK, 0 rows affected (0.0172 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL> insert into insurance (vehicle_num, provider, expiry_date, policy_description) values
-> ('AP001','bajaj allianz','2026-10-31','comprehensive policy'),
-> ('AP002','royal sundaram','2026-07-20','third-party + add-on'),
-> ('AP003','icici lombard','2026-08-15','comprehensive'),
-> ('AP004','tata aig','2027-01-10','commercial vehicle policy'),
-> ('AP005','hdfc ergo','2026-11-12','fleet insurance'),
-> ('AP006','new india assurance','2026-06-30','two-wheeler comprehensive'),
-> ('AP007','reliance general','2026-09-05','fleet comprehensive'),
-> ('AP008','bajaj allianz','2026-12-01','comprehensive'),
-> ('AP009','nikunj insurance','2026-05-20','third-party'),
-> ('AP010','royal sundaram','2027-02-18','commercial van policy'),
-> ('AP011','icici lombard','2026-04-25','e-scooter policy'),
-> ('AP012','tata aig','2026-03-30','ev car policy'),
-> ('AP013','hdfc ergo','2026-07-07','two-wheeler'),
-> ('AP014','new india assurance','2026-09-15','commuter bus'),
-> ('AP015','reliance general','2026-10-05','commercial'),
-> ('AP016','bajaj allianz','2026-08-22','two-wheeler'),
-> ('AP017','royal sundaram','2027-01-30','creta suv policy'),
-> ('AP018','icici lombard','2026-06-18','etios policy'),
-> ('AP019','tata aig','2026-11-01','cargo van policy'),
-> ('AP020','hdfc ergo','2026-12-12','mini truck policy');

Query OK, 20 rows affected (0.0042 sec)

Records: 20 Duplicates: 0 Warnings: 0

MySQL localhost:33060+ ssl vehicle_rental_system SQL> create table feedback (
-> feedback_id int primary key auto_increment,
-> booking_id int not null,
-> customer_id int not null,
-> rating int check (rating between 1 and 5),
-> comments text,
-> foreign key (booking_id) references booking(booking_id),
-> foreign key (customer_id) references customers(cust_id)
-> );

Query OK, 0 rows affected (0.0217 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL> insert into feedback (booking_id, customer_id, rating, comments) values
-> (1,1,5,'excellent service, clean car'),
-> (2,2,4,'bike was fine, slight delay in pickup'),
-> (3,3,4,'good overall'),
-> (4,4,5,'innova was comfortable for family trip'),
-> (5,5,3,'ac was weak on day 2'),
-> (6,6,5,'very satisfied with service'),
-> (7,7,4,'nice interiors'),
-> (8,8,3,'vehicle had minor issues'),
-> (9,9,4,'timely pickup and drop'),
-> (10,10,5,'great driver support'),
-> (11,11,4,'smooth process'),
-> (12,12,5,'ev experience was great'),
-> (13,13,3,'expected better cleanliness'),
-> (14,14,5,'tempo traveller perfect for group'),
-> (15,15,4,'bolero was sturdy'),
-> (16,16,4,'quick service'),
-> (17,17,5,'creta drove smoothly'),
-> (18,18,3,'minor scratches but worked fine'),
-> (19,19,4,'cargo van did the job'),
-> (20,20,5,'mini truck delivery was prompt');

Query OK, 20 rows affected (0.0033 sec)

Records: 20 Duplicates: 0 Warnings: 0

```


6. Database Programming

Stored Procedures

Many procedures can be created from the database we have created, two main of them are here.

1. add_customer()

```
delimiter $$
create procedure add_customer(
    in p_name varchar(100),
    in p_phone varchar(20),
    in p_mail varchar(255),
    in p_street varchar(120),
    in p_city varchar(80),
    in p_state varchar(80),
    in p_pincode varchar(10),
    in p_license_no varchar(50)
)
begin
    insert into customers (name, phone_num, mail_id, street, city,
state, pincode, license_no)
    values (p_name, p_phone, p_mail, p_street, p_city, p_state,
p_pincode, p_license_no);
end $$
delimiter ;
```

Testing query:

```
call add_customer(
    'deepak nair', '9876501234', 'deepak.nair@gmail.com', '7-2-15',
    'mangalore', 'karnataka', '575001', 'KA05L99999'
);

select * from customers where name = 'deepak nair';
```

Stored Procedures

2. add_booking()

```
delimiter $$
create procedure add_booking(
  in p_cust_id int,
  in p_vehicle_num varchar(32),
  in p_start_date date,
  in p_start_time time,
  in p_end_date date,
  in p_end_time time,
  in p_pay_amount decimal(12,2)
)
begin
  insert into booking (cust_id, vehicle_num, start_date, start_time,
end_date, end_time, pay_amount)
  values (p_cust_id, p_vehicle_num, p_start_date, p_start_time,
p_end_date, p_end_time, p_pay_amount);

  -- marking the vehicle as unavailable after booking
  update vehicles set avail_status = 'unavailable' where
vehicle_num = p_vehicle_num;
end $$
delimiter ;
```

Testing query:

```
call add_booking(18, 'AP002', '2025-11-10', '09:00:00', '2025-11-12',
'09:00:00', 3600.00);

select * from booking where cust_id = 18;
select vehicle_num, avail_status from vehicles where vehicle_num
= 'AP002';
```

Stored Procedures

1. add_customer()

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter $$
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create procedure add_customer(
    -> in p_name varchar(100),
    -> in p_phone varchar(20),
    -> in p_mail varchar(255),
    -> in p_street varchar(120),
    -> in p_city varchar(80),
    -> in p_state varchar(80),
    -> in p_pincode varchar(10),
    -> in p_license_no varchar(50)
    -> )
    -> begin
    -> insert into customers (name, phone_num, mail_id, street, city,
    -> state, pincode, license_no)
    -> values (p_name, p_phone, p_mail, p_street, p_city, p_state,
    -> p_pincode, p_license_no);
    -> end $$

Query OK, 0 rows affected (0.0285 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter ;
MySQL localhost:33060+ ssl vehicle_rental_system SQL > call add_customer(
    -> 'deepak nair', '9876501234', 'deepak.nair@gmail.com', '7-2-15',
    -> 'mangalore', 'karnataka', '575001', 'KA05L99999'
    -> );

Query OK, 1 row affected (0.0349 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from customers where name = 'deepak nair';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| cust_id | name      | phone_num | mail_id      | street | city      | state    | pincode | license_no |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 21      | deepak nair | 9876501234 | deepak.nair@gmail.com | 7-2-15 | mangalore | karnataka | 575001 | KA05L99999 |
+-----+-----+-----+-----+-----+-----+-----+-----+

1 row in set (0.0014 sec)
```

2. add_booking()

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter $$
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create procedure add_booking(
    -> in p_cust_id int,
    -> in p_vehicle_num varchar(32),
    -> in p_start_date date,
    -> in p_start_time time,
    -> in p_end_date date,
    -> in p_end_time time,
    -> in p_pay_amount decimal(12,2)
    -> )
    -> begin
    -> insert into booking (cust_id, vehicle_num, start_date, start_time, end_date, end_time, pay_amount)
    -> values (p_cust_id, p_vehicle_num, p_start_date, p_start_time, p_end_date, p_end_time, p_pay_amount);
    ->
    -> -- marking the vehicle as unavailable after booking
    -> update vehicles set avail_status = 'unavailable' where vehicle_num = p_vehicle_num;
    -> end $$

Query OK, 0 rows affected (0.0020 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter ;
MySQL localhost:33060+ ssl vehicle_rental_system SQL > call add_booking(18, 'AP002', '2025-11-10', '09:00:00', '2025-11-12', '09:00:00', 3600.00);

Query OK, 1 row affected (0.0035 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from booking where cust_id = 18;
+-----+-----+-----+-----+-----+-----+-----+
| booking_id | cust_id | vehicle_num | start_date | start_time | end_date | end_time | pay_amount |
+-----+-----+-----+-----+-----+-----+-----+
| 18         | 18      | AP018       | 2025-10-30 | 08:10:00   | 2025-11-01 | 08:10:00 | 3300.00    |
| 22         | 18      | AP002       | 2025-11-10 | 09:00:00   | 2025-11-12 | 09:00:00 | 3600.00    |
+-----+-----+-----+-----+-----+-----+-----+

2 rows in set (0.0005 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select vehicle_num, avail_status from vehicles where vehicle_num = 'AP002';
+-----+-----+
| vehicle_num | avail_status |
+-----+-----+
| AP002       | unavailable   |
+-----+-----+

1 row in set (0.0009 sec)
```

User-Defined Function

Many user-defined functions can be created from the database we have created, two main of them are here.

1. get_rental_days()

```
delimiter $$  
create function get_rental_days(p_booking_id int)  
returns int  
deterministic  
begin  
    declare v_start date;  
    declare v_end date;  
    declare v_days int;  
  
    select start_date, end_date  
    into v_start, v_end  
    from booking  
    where booking_id = p_booking_id;  
  
    set v_days = datediff(v_end, v_start);  
    return v_days;  
end $$  
delimiter ;
```

The function get_rental_days() calculates the number of rental days for a given booking by finding the date difference between the start and end dates.

Testing query:

```
select get_rental_days(15) as rental_days;
```

User-Defined Function

2. calculate_total_amount()

```
delimiter $$
create function calculate_total_amount(p_booking_id int)
returns decimal(10,2)
deterministic
begin
    declare v_rent decimal(10,2);
    declare v_penalty decimal(10,2);
    declare v_total decimal(10,2);

    select b.pay_amount, ifnull(bl.penalty_amount, 0)
    into v_rent, v_penalty
    from booking b
    left join billing bl on b.booking_id = bl.booking_id
    where b.booking_id = p_booking_id;

    set v_total = v_rent + v_penalty;

    return v_total;
end $$
delimiter ;
```

The function calculate_total_amount() returns the final payable amount for a given booking by adding the base rent and any penalty amount (if applicable).

Testing query:

```
select calculate_total_amount(15);
```

User-Defined Function

1. get_rental_days()

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter $$
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create function get_rental_days(p_booking_id int)
-> returns int
-> deterministic
-> begin
->     declare v_start date;
->     declare v_end date;
->     declare v_days int;
->     select start_date, end_date
->     into v_start, v_end
->     from booking
->     where booking_id = p_booking_id;
->     set v_days = datediff(v_end, v_start);
->     return v_days;
-> end $$
Query OK, 0 rows affected (0.0053 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter ;
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select get_rental_days(15) as rental_days;
+-----+
| rental_days |
+-----+
|          2 |
+-----+
1 row in set (0.0097 sec)
```

2. calculate_total_amount()

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter $$
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create function calculate_total_amount(p_booking_id int)
-> returns decimal(10,2)
-> deterministic
-> begin
->     declare v_rent decimal(10,2);
->     declare v_penalty decimal(10,2);
->     declare v_total decimal(10,2);
->     select b.pay_amount, ifnull(bl.penalty_amount, 0)
->     into v_rent, v_penalty
->     from booking b
->     left join billing bl on b.booking_id = bl.booking_id
->     where b.booking_id = p_booking_id;
->     set v_total = v_rent + v_penalty;
->     return v_total;
-> end $$
Query OK, 0 rows affected (0.0031 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter ;
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select calculate_total_amount(15);
+-----+
| calculate_total_amount(15) |
+-----+
|          5000.00 |
+-----+
1 row in set (0.0027 sec)
```

Triggers

1. before_insert_booking

```
delimiter $$
create trigger before_insert_booking
before insert on booking
for each row
begin
    declare v_status varchar(20);
    select avail_status into v_status
    from vehicles
    where vehicle_num = new.vehicle_num;

    if v_status != 'available' then
        signal sqlstate '45000'
        set message_text = 'vehicle is not available for booking';
    end if;
end $$
delimiter ;
```

1. Trigger: before_insert_booking

This trigger runs before inserting a new booking.

It checks the availability status of the vehicle being booked.

- If avail_status != 'available'
- → The trigger blocks the booking
- → Throws an error: "vehicle is not available for booking"

```
update vehicles
set avail_status = 'unavailable'
where vehicle_num = 'AP018';
insert into booking
(booking_id, cust_id, vehicle_num, start_date, start_time,
end_date, end_time, pay_amount)
values
(22, 15, 'AP018', '2025-04-01', '10:00:00', '2025-04-02', '10:00:00',
1200.00);
```

The insert query should fail working, throws an error due to the trigger.

Triggers

2. after_maintenance_insert

```
delimiter $$
create trigger after_maintenance_insert
after insert on maintainance
for each row
begin
    update vehicles
    set avail_status = 'in_service'
    where vehicle_num = new.vehicle_num;
end $$
delimiter ;
```

Trigger: after_maintenance_insert

This trigger runs after a new maintenance record is inserted.

- It automatically changes the vehicle's status to 'in_service'.

Purpose:

Ensures the system automatically marks the vehicle unavailable for booking while maintenance is ongoing.

```
insert into maintainance
(maintenance_id, vehicle_num, service, service_date, service_time,
expenditure, emp_id)
values
(21, 'AP005', 'oil change', '2025-03-20', '11:00:00', 500.00, 1);

select vehicle_num, avail_status
from vehicles
where vehicle_num = 'AP005';
```

Expected Result

vehicle_num	avail_status
AP005	in_service

Triggers

1. before_insert_booking

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter $$
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create trigger before_insert_booking
-> before insert on booking
-> for each row
-> begin
->     declare v_status varchar(20);
->     select avail_status into v_status
->     from vehicles
->     where vehicle_num = new.vehicle_num;
->
->     if v_status != 'available' then
->         signal sqlstate '45000'
->         set message_text = 'vehicle is not available for booking';
->     end if;
-> end $$
Query OK, 0 rows affected (0.0044 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter ;
MySQL localhost:33060+ ssl vehicle_rental_system SQL > update vehicles
-> set avail_status = 'unavailable'
-> where vehicle_num = 'AP018';
Query OK, 1 row affected (0.0014 sec)
Rows matched: 1  Changed: 1  Warnings: 0
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into booking
-> (booking_id, cust_id, vehicle_num, start_date, start_time, end_date, end_time, pay_amount)
-> values
-> (22, 15, 'AP018', '2025-04-01', '10:00:00', '2025-04-02', '10:00:00', 1200.00);
ERROR: 1644: vehicle is not available for booking
```

2. after_maintenance_insertnce

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter $$
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create trigger after_maintenance_insert
-> after insert on maintainance
-> for each row
-> begin
->     update vehicles
->     set avail_status = 'in_service'
->     where vehicle_num = new.vehicle_num;
-> end $$
Query OK, 0 rows affected (0.0036 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > delimiter ;
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into maintainance
-> (maintainance_id, vehicle_num, service, service_date, service_time, expenditure, emp_id)
-> values
-> (21, 'AP005', 'oil change', '2025-03-20', '11:00:00', 500.00, 1);
Query OK, 1 row affected (0.0025 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL >
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select vehicle_num, avail_status
-> from vehicles
-> where vehicle_num = 'AP005';
+-----+-----+
| vehicle_num | avail_status |
+-----+-----+
| AP005      | in_service   |
+-----+-----+
1 row in set (0.0007 sec)
```

Transaction Control

Start a new transaction. and insert booking details into booking table.

```
start transaction;  
  
insert into booking (cust_id, vehicle_num, start_date, start_time,  
end_date, end_time, pay_amount)  
values (3, 'AP010', '2025-11-15', '10:00:00', '2025-11-17', '09:00:00',  
2400.00);
```

Save the current point in the transaction

```
select * from booking order by booking_id desc limit 3;  
  
savepoint booking_created;
```

Insert an entry into payment, as if you are adding a new payment

```
insert into payment (billing_id, booking_id, payment_mode,  
payment_for, payment_date, payment_time, amount_paid)  
values (null, (select max(booking_id) from booking), 'upi', 'advance',  
curdate(), curtime(), 2400.00);  
  
select * from payment order by payment_id desc limit 3;
```

Rollback ONLY to savepoint

```
rollback to booking_created;
```

Transaction Control

Checking both tables now

```
select * from booking order by booking_id desc limit 3;
select * from payment order by payment_id desc limit 3;
```

Commit the transaction

```
commit;
```

Testing both tables

```
select * from booking where booking_id = <your_new_id>;
select * from payment order by payment_id desc limit 3;
```

Final optional rollback

```
rollback;
```

Execution in MySQL

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > start transaction;
Query OK, 0 rows affected (0.0005 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into booking (cust_id, vehicle_num, start_date, start_time, end_date, end_time, pay_amount)
-> values (3, 'AP010', '2025-11-15', '10:00:00', '2025-11-17', '09:00:00', 2400.00);
Query OK, 1 row affected (0.0008 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from booking order by booking_id desc limit 3;
+-----+-----+-----+-----+-----+-----+-----+
| booking_id | cust_id | vehicle_num | start_date | start_time | end_date | end_time | pay_amount |
+-----+-----+-----+-----+-----+-----+-----+
| 504        | 3       | AP010       | 2025-11-15 | 10:00:00   | 2025-11-17 | 09:00:00 | 2400.00    |
| 503        | 3       | AP010       | 2025-11-15 | 10:00:00   | 2025-11-17 | 09:00:00 | 2400.00    |
| 22         | 18      | AP002       | 2025-11-10 | 09:00:00   | 2025-11-12 | 09:00:00 | 3600.00    |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.0009 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > savepoint booking_created;
Query OK, 0 rows affected (0.0006 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > insert into payment (billing_id, booking_id, payment_mode, payment_for, payment_date, payment_time, amount_paid)
-> values (null, (select max(booking_id) from booking), 'upi', 'advance', curdate(), curtime(), 2400.00);
Query OK, 1 row affected (0.0010 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from payment order by payment_id desc limit 3;
+-----+-----+-----+-----+-----+-----+-----+
| payment_id | billing_id | booking_id | payment_mode | payment_for | payment_date | payment_time | amount_paid |
+-----+-----+-----+-----+-----+-----+-----+
| 43         | NULL       | 504        | upi          | advance     | 2025-11-14   | 16:34:03    | 2400.00    |
| 42         | NULL       | 503        | upi          | advance     | 2025-11-14   | 16:17:24    | 2400.00    |
| 40         | 20         | 20         | card        | penalty     | 2025-11-05   | 14:35:00    | 600.00     |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.0010 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > rollback to booking_created;
Query OK, 0 rows affected (0.0009 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from booking order by booking_id desc limit 3;
+-----+-----+-----+-----+-----+-----+-----+
| booking_id | cust_id | vehicle_num | start_date | start_time | end_date | end_time | pay_amount |
+-----+-----+-----+-----+-----+-----+-----+
| 504        | 3       | AP010       | 2025-11-15 | 10:00:00   | 2025-11-17 | 09:00:00 | 2400.00    |
| 503        | 3       | AP010       | 2025-11-15 | 10:00:00   | 2025-11-17 | 09:00:00 | 2400.00    |
| 22         | 18      | AP002       | 2025-11-10 | 09:00:00   | 2025-11-12 | 09:00:00 | 3600.00    |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.0005 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from payment order by payment_id desc limit 3;
+-----+-----+-----+-----+-----+-----+-----+
| payment_id | billing_id | booking_id | payment_mode | payment_for | payment_date | payment_time | amount_paid |
+-----+-----+-----+-----+-----+-----+-----+
| 42         | NULL       | 503        | upi          | advance     | 2025-11-14   | 16:17:24    | 2400.00    |
| 40         | 20         | 20         | card        | penalty     | 2025-11-05   | 14:35:00    | 600.00     |
| 39         | 19         | 19         | upi          | penalty     | 2025-11-04   | 09:25:00    | 800.00     |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.0002 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > |
```

7. Query Design

Data Retrieval using Joins and Subqueries

Query: Retrieve all booking details with customer names, vehicle models, and company names.

```
select
  c.name as customer_name,
  b.booking_id,
  v.vehicle_num,
  vm.model,
  co.com_name as company,
  b.start_date,
  b.end_date,
  b.pay_amount
from booking b
join customers c on b.cust_id = c.cust_id
join vehicles v on b.vehicle_num = v.vehicle_num
join vehicle_model vm on v.vehicle_id = vm.vehicle_id
join company co on vm.company_id = co.com_id
where b.cust_id in (
  select cust_id
  from booking
  group by cust_id
  having count(booking_id) > 1
)
order by c.name;
```

7. Query Design

Aggregation and Grouping (GROUP BY, HAVING)

Query: Find the total number of bookings and total revenue per vehicle model. Display only those models with total revenue greater than ₹10,000.

```
select
  vm.model,
  count(b.booking_id) as total_bookings,
  sum(b.pay_amount) as total_revenue
from booking b
join vehicles v on b.vehicle_num = v.vehicle_num
join vehicle_model vm on v.vehicle_id = vm.vehicle_id
group by vm.model
having sum(b.pay_amount) > 10000
order by total_revenue desc;
```

Data Modification (INSERT, UPDATE, DELETE)

(a) INSERT Example: Add a new customer

```
insert into customers (name, phone_num, mail_id, street, city, state,
pincode, license_no)
values ('ranjit das', '9876543200', 'ranjit.das@gmail.com', '22-3-15',
'kolkata', 'west bengal', '700005', 'WB20L45123');
```

(b) UPDATE Example: Update vehicle status after return

```
update vehicles
set avail_status = 'available'
where vehicle_num = 'AP003';
```

(c) DELETE Example: Delete old maintenance records before a given date

```
delete from maintainance
where service_date < '2025-01-01';
```

7. Query Design

Analytical Reporting (Top Performers & Monthly Revenue)

(a) Top 5 Customers by Total Spending

```
select
    c.name as customer_name,
    sum(p.amount_paid) as total_spent
from customers c
join booking b on c.cust_id = b.cust_id
join payment p on p.booking_id = b.booking_id
group by c.name
order by total_spent desc
limit 5;
```

(b) Monthly Revenue Report (2025)

```
select
    year(payment_date) as year,
    monthname(payment_date) as month,
    sum(amount_paid) as total_revenue
from payment
where year(payment_date) = 2025
group by year(payment_date), month(payment_date)
order by month(payment_date);
```

Optimization using Views

Create a View: Pre-compute customer booking summary for quick reporting.

```
create view customer_booking_summary as
select
    c.cust_id,
    c.name as customer_name,
    count(b.booking_id) as total_bookings,
    sum(b.pay_amount) as total_booking_amount,
    sum(ifnull(bl.total_amount, 0)) as total_billed_amount
from customers c
left join booking b on c.cust_id = b.cust_id
left join billing bl on bl.booking_id = b.booking_id
group by c.cust_id, c.name;
```

7. Query Design

Data Retrieval using Joins and Subqueries

Query: Retrieve all booking details with customer names, vehicle models, and company names.

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL> select
-> c.name as customer_name,
-> b.booking_id,
-> v.vehicle_num,
-> vm.model,
-> co.com_name as company,
-> b.start_date,
-> b.end_date,
-> b.pay_amount
-> from booking b
-> join customers c on b.cust_id = c.cust_id
-> join vehicles v on b.vehicle_num = v.vehicle_num
-> join vehicle_model vm on v.vehicle_id = vm.vehicle_id
-> join company co on vm.company_id = co.com_id
-> where b.cust_id in (
-> select cust_id
-> from booking
-> group by cust_id
-> having count(booking_id) > 1
-> )
-> order by c.name;
```

customer_name	booking_id	vehicle_num	model	company	start_date	end_date	pay_amount
kavya menon	18	AP018	etios liva	toyota	2025-10-30	2025-11-01	3300.00
kavya menon	22	AP002	activa 6g	honda motorcycles	2025-11-10	2025-11-12	3600.00

2 rows in set (0.0563 sec)

Aggregation and Grouping (GROUP BY, HAVING)

Query: Find the total number of bookings and total revenue per vehicle model. Display only those models with total revenue greater than ₹10,000.

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL> select
-> vm.model,
-> count(b.booking_id) as total_bookings,
-> sum(b.pay_amount) as total_revenue
-> from booking b
-> join vehicles v on b.vehicle_num = v.vehicle_num
-> join vehicle_model vm on v.vehicle_id = vm.vehicle_id
-> group by vm.model
-> having sum(b.pay_amount) > 10000
-> order by total_revenue desc;
```

model	total_bookings	total_revenue
tempo traveller 12 seater	1	12600.00
creta sport edition	1	10200.00

2 rows in set (0.0014 sec)

Data Modification (INSERT, UPDATE, DELETE)

(a) INSERT Example: Add a new customer

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL> insert into customers (name, phone_num, mail_id, street, city, state, pincode, license_no) values ('ranjith das', '9876543100', 'ranjith.das@gmail.com', '22-3-15', 'kolkata', 'west bengal', '700005', 'WB20L43124');
Query OK, 1 row affected (0.0089 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL> select * from customers where name='ranjith das';
```

cust_id	name	phone_num	mail_id	street	city	state	pincode	license_no
26	ranjith das	9876543100	ranjith.das@gmail.com	22-3-15	kolkata	west bengal	700005	WB20L43124

1 row in set (0.0014 sec)

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL> update vehicles
-> set avail_status = 'available'
-> where vehicle_num = 'AP003';
Query OK, 1 row affected (0.0021 sec)

Rows matched: 1 Changed: 0 Warnings: 0
```


7. Query Design

(b) UPDATE Example: Update vehicle status after return

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > update vehicles
-> set avail_status = 'available'
-> where vehicle_num = 'AP003';

Query OK, 1 row affected (0.0021 sec)

Rows matched: 1 Changed: 0 Warnings: 0
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select avail_status from vehicles where vehicle_num='AP003';
+-----+
| avail_status |
+-----+
| available    |
+-----+
1 row in set (0.0012 sec)
```

(c) DELETE Example: Delete old maintenance records before a given date

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from maintainance;
+-----+-----+-----+-----+-----+-----+-----+
| maintainance_id | vehicle_num | service           | service_date | service_time | expenditure | emp_id |
+-----+-----+-----+-----+-----+-----+-----+
| 1               | AP001       | oil change        | 2025-09-10   | 10:00:00     | 1200.00     | 5      |
| 2               | AP002       | brake check       | 2025-09-12   | 11:30:00     | 800.00      | 2      |
| 3               | AP003       | tyre replacement  | 2025-09-15   | 09:00:00     | 4000.00     | 5      |
| 4               | AP004       | engine service    | 2025-09-20   | 08:30:00     | 8500.00     | 11     |
| 5               | AP005       | ac repair         | 2025-09-22   | 10:15:00     | 3000.00     | 17     |
| 6               | AP006       | battery change    | 2025-09-25   | 14:00:00     | 6000.00     | 16     |
| 7               | AP007       | interior cleaning | 2025-09-28   | 09:45:00     | 1200.00     | 6      |
| 8               | AP008       | gearbox repair    | 2025-09-30   | 10:30:00     | 9000.00     | 5      |
| 9               | AP009       | tyre rotation     | 2025-10-02   | 11:00:00     | 600.00      | 2      |
| 10              | AP010       | wheel alignment   | 2025-10-05   | 09:20:00     | 700.00      | 5      |
| 11              | AP011       | software update   | 2025-10-08   | 12:00:00     | 500.00      | 18     |
| 12              | AP012       | full service      | 2025-10-10   | 08:00:00     | 5500.00     | 11     |
| 13              | AP013       | chain lubrication | 2025-10-12   | 09:10:00     | 300.00      | 2      |
| 14              | AP014       | ac gas refill     | 2025-10-14   | 10:40:00     | 1500.00     | 8      |
| 15              | AP015       | suspension check  | 2025-10-16   | 11:25:00     | 1200.00     | 17     |
| 16              | AP016       | spark plug change | 2025-10-18   | 08:50:00     | 400.00      | 5      |
| 17              | AP017       | filter replacement | 2025-10-20   | 09:30:00     | 900.00      | 5      |
| 18              | AP018       | battery check     | 2025-10-22   | 10:10:00     | 350.00      | 16     |
| 19              | AP019       | cargo door repair | 2025-10-24   | 11:40:00     | 2200.00     | 11     |
| 20              | AP020       | coolant refill    | 2025-10-26   | 09:00:00     | 800.00      | 5      |
+-----+-----+-----+-----+-----+-----+-----+
20 rows in set (0.0017 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL > delete from maintainance
-> where service_date < '2025-09-22';

Query OK, 4 rows affected (0.0089 sec)

MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from maintainance;
+-----+-----+-----+-----+-----+-----+-----+
| maintainance_id | vehicle_num | service           | service_date | service_time | expenditure | emp_id |
+-----+-----+-----+-----+-----+-----+-----+
| 5               | AP005       | ac repair         | 2025-09-22   | 10:15:00     | 3000.00     | 17     |
| 6               | AP006       | battery change    | 2025-09-25   | 14:00:00     | 6000.00     | 16     |
| 7               | AP007       | interior cleaning | 2025-09-28   | 09:45:00     | 1200.00     | 6      |
| 8               | AP008       | gearbox repair    | 2025-09-30   | 10:30:00     | 9000.00     | 5      |
| 9               | AP009       | tyre rotation     | 2025-10-02   | 11:00:00     | 600.00      | 2      |
| 10              | AP010       | wheel alignment   | 2025-10-05   | 09:20:00     | 700.00      | 5      |
| 11              | AP011       | software update   | 2025-10-08   | 12:00:00     | 500.00      | 18     |
| 12              | AP012       | full service      | 2025-10-10   | 08:00:00     | 5500.00     | 11     |
| 13              | AP013       | chain lubrication | 2025-10-12   | 09:10:00     | 300.00      | 2      |
| 14              | AP014       | ac gas refill     | 2025-10-14   | 10:40:00     | 1500.00     | 8      |
| 15              | AP015       | suspension check  | 2025-10-16   | 11:25:00     | 1200.00     | 17     |
| 16              | AP016       | spark plug change | 2025-10-18   | 08:50:00     | 400.00      | 5      |
| 17              | AP017       | filter replacement | 2025-10-20   | 09:30:00     | 900.00      | 5      |
| 18              | AP018       | battery check     | 2025-10-22   | 10:10:00     | 350.00      | 16     |
| 19              | AP019       | cargo door repair | 2025-10-24   | 11:40:00     | 2200.00     | 11     |
| 20              | AP020       | coolant refill    | 2025-10-26   | 09:00:00     | 800.00      | 5      |
+-----+-----+-----+-----+-----+-----+-----+
16 rows in set (0.0017 sec)
```

Analytical Reporting (Top Performers & Monthly Revenue)

(a) Top 5 Customers by Total Spending

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select
-> c.name as customer_name,
-> sum(p.amount_paid) as total_spent
-> from customers c
-> join booking b on c.cust_id = b.cust_id
-> join payment p on p.booking_id = b.booking_id
-> group by c.name
-> order by total_spent desc
-> limit 5;

+-----+-----+
| customer_name | total_spent |
+-----+-----+
| rahul sharma  | 14600.00   |
| soumya reddy  | 11700.00   |
| deeksha reddy | 11000.00   |
| revanth varma | 9400.00    |
| ananya verma  | 8600.00    |
+-----+-----+
5 rows in set (0.0070 sec)
```


7. Query Design

(b) Monthly Revenue Report (2025)

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select
->     year(payment_date) as year,
->     monthname(payment_date) as month,
->     month(payment_date) as month_no,
->     sum(amount_paid) as total_revenue
-> from payment
-> where year(payment_date) = 2025
-> group by year, month, month_no
-> order by month_no;
```

year	month	month_no	total_revenue
2025	October	10	77010.00
2025	November	11	35140.00

2 rows in set (0.0014 sec)

Optimization using Views

Create a View: Pre-compute customer booking summary for quick reporting.

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > drop view customer_booking_summary;
Query OK, 0 rows affected (0.0050 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > create view customer_booking_summary as
-> select
->     c.cust_id,
->     c.name as customer_name,
->     count(b.booking_id) as total_bookings,
->     sum(b.pay_amount) as total_booking_amount,
->     sum(ifnull(bl.total_amount, 0)) as total_billed_amount
-> from customers c
-> left join booking b on c.cust_id = b.cust_id
-> left join billing bl on bl.booking_id = b.booking_id
-> group by c.cust_id, c.name;
Query OK, 0 rows affected (0.0045 sec)
MySQL localhost:33060+ ssl vehicle_rental_system SQL > select * from customer_booking_summary;
```

cust_id	customer_name	total_bookings	total_booking_amount	total_billed_amount
1	arjun reddy	1	5400.00	6400.00
2	priya sharma	1	800.00	1000.00
3	rahul kumar	1	900.00	1200.00
4	ananya verma	1	7000.00	8600.00
5	sai kiran	1	6000.00	7400.00
6	isha patel	1	2100.00	2700.00
7	vinit mehta	1	4400.00	4900.00
8	deeksha reddy	1	9600.00	11000.00
9	karan singh	1	2700.00	3200.00
10	madhu priya	1	3000.00	3800.00
11	sachin naik	1	500.00	650.00
12	revanth varma	1	7600.00	9400.00
13	varsha gowda	1	2400.00	3000.00
14	rahul sharma	1	12600.00	14600.00
15	meghana joshi	1	4000.00	5000.00
16	aditya jain	1	1100.00	1340.00
17	soumya reddy	1	10200.00	11700.00
18	kavya menon	2	6900.00	4200.00
19	naveen ch	1	3200.00	4000.00
20	sanju yadav	1	5000.00	5600.00
21	deepak nair	0	NULL	0.00
22	ranjit das	0	NULL	0.00
26	ranjith das	0	NULL	0.00

23 rows in set (0.0018 sec)

```
MySQL localhost:33060+ ssl vehicle_rental_system SQL > |
```

8. Access Control

1. Admin Role

The Admin user has full privileges on the entire database, including the ability to create tables, modify records, run procedures, and manage other users.

```
-- create admin user
create user 'admin_user'@'localhost' identified by
'Admin@123';

-- grant full privileges on the entire database
grant all privileges on vehicle_rental_db.* to
'admin_user'@'localhost';

-- apply changes
flush privileges;
```

2. Normal User Role

The User account has restricted privileges.

They can only perform read-only queries such as viewing available vehicles, bookings, or payment history.

They cannot insert, update, delete, or modify schema.

```
-- create normal user
create user 'app_user'@'localhost' identified by
'User@123';

-- grant read-only privileges
grant select on vehicle_rental_db.* to
'app_user'@'localhost';

-- apply changes
flush privileges;
```

3. Example of Access Revocation

```
revoke insert, update, delete on vehicle_rental_db.* from
'app_user'@'localhost';
flush privileges;
```

9. Conclusion and Future Scope

Conclusion

The Vehicle Rental Management System successfully demonstrates how a well-designed database can streamline and automate rental operations. The project covers customer management, vehicle details, bookings, billing, payments, maintenance, and employee information with a structured and normalized database design. By implementing primary and foreign keys, constraints, stored procedures, triggers, and user-defined functions, the system ensures consistency, accuracy, and reliability of data. Normalization up to 3NF helped remove redundancy and improved the overall efficiency of data storage.

Overall, the system provides a strong and scalable backend capable of supporting essential vehicle rental operations in a secure and organized manner.

Future Scope

The system can be enhanced by integrating a user-friendly mobile or web application to enable online bookings, payments, and customer interactions. Features like GPS-based vehicle tracking, automated billing and invoice generation, and notification alerts for insurance renewal or maintenance can further improve usability. Implementing role-based access control will strengthen security. Additionally, cloud deployment and analytics dashboards can be added to support scalability, generate insights, and assist management in decision-making.

These improvements can turn the system into a complete, real-world vehicle rental management solution.

10. Individual Contributions

Chaitanya Naga Sai Pavan S20240010237

- ER Diagram Designing
- Normalization
- Transaction Control
- Primary and Foreign keys implementation and constraints

Thota Prem Kumar S20240010243

- Database Programming (Procedures, Triggers)
- Execution of the Queries, Procedures, Triggers and Functions
- Helped in ER Diagram Designing
- Justification of selected users

Shameer Basha Shaik S20240010222

- PDF Designing
- Helped in making ER Diagram and Transaction Controls
- Query Designing
- Database Programming(Functions)
- Requirement Analysis, Intro and Objectives

Hemanth Kumar Avunuri S20240010018

- Data Population
- Helped in logical Database Design
- Helped in Access Controls
- Conclusion

Gunda Naga Venkata Ghana Sathwik
S20240030370

- Logical Design of the database
- Helped in Normalization
- Access Controls
- Future Scope